Pure-C Transformer Engine

The Complete Guide

Build - Train - Fine-tune - Reinforce - Deploy - Embed

A GPT-2 / LLaMA-family transformer written from scratch in portable C11: no BLAS, no CUDA toolkit, no Python at runtime, and no third-party headers anywhere in the tree. It trains, fine-tunes with LoRA, reinforces with GRPO, quantizes to int8, runs a tool-using agent, and now optionally offloads its matrix multiplies to a GPU through an OpenCL driver it loads at runtime.

This guide covers every step end to end: compiling each build flavour, preparing data, pre-training, supervised fine-tuning, LoRA, GRPO reinforcement learning, GPU acceleration, running on Google Colab, and linking the engine into your own C program. Every command-line parameter is documented with its default, its effect, and when you would actually reach for it.

Read this first

Section 3 gets you a working binary. Section 6 trains a model. Section 11 links it into your own program. If you only have ten minutes, read 3.1, 6.2 and 11.3.

Version: engine @ agent-1 (GRPO + OpenCL GPU backend)
Generated: 2 September 2026
Generated by tools/guidepdf (Go + gofpdf). Regenerate with: cd tools/guidepdf && go run .

Contents

1 What this engine is 4
- 1.1 The layout rule 4
- 1.2 Module map 4
- 1.3 What it can and cannot do 5

2 Prerequisites and toolchain 6
- 2.1 Verifying the toolchain 6
- 2.2 Getting the sources 6

3 Building 7
- 3.1 The CPU build (start here) 7
- 3.2 The GPU build 7
- 3.3 Cross-compiling 8
- 3.4 Verifying a build 8

4 Command reference 9
- 4.1 Global flags 9
- 4.2 generate, llama, hfgen, hfchat 9
- 4.3 agent 10
- 4.4 gpu-info and gpu-bench 10

5 Preparing data 11
- 5.1 Plain text (pre-training) 11
- 5.2 prompt TAB response (instruction tuning) 11
- 5.3 Tools for building corpora 11

6 Training, step by step 13
- 6.1 Which trainer to use 13
- 6.2 Walkthrough: train a model from scratch 13
- 6.3 Every training parameter 13
- 6.4 Supervised fine-tuning (SFT) 14
- 6.5 LoRA fine-tuning 15
- 6.6 Reading the output 15

7 DPO: learning from preferences 16
- 7.1 Why it exists alongside GRPO 16
- 7.2 The objective 16
- 7.3 Walkthrough 16
- 7.4 Every DPO parameter 17
- 7.5 Where the rejected side comes from 17
- 7.6 Measured cost, and the honest comparison 17
- 7.7 Honest expectations 18

8 GRPO: reinforcement learning from verifiable rewards 19
- 8.1 How it works 19
- 8.2 Walkthrough 19
- 8.3 Every GRPO parameter 19
- 8.4 The reward 20
- 8.5 Two behaviours worth knowing 21

9 GPU acceleration 22
- 9.1 How it hooks in 22
- 9.2 When the GPU is used 22
- 9.3 Deciding whether to use it 22
- 9.4 Correctness 23

9.5 Known limitation 23

10 Running on Google Colab 24

10.1 Step 1 - build the binary 24
10.2 Step 2 - open the notebook and pick a GPU 24
10.3 Step 3 - upload the binary 24
10.4 Step 4 - install an OpenCL loader 24
10.5 Step 5 - verify, then measure 25
10.6 Step 6 - a real workload 25

11 Loading a model in your own program 26

11.1 The pieces you need 26
11.2 The four steps 26
11.3 A complete, working example 26
11.4 Memory and performance notes 28
11.5 Chat models 28

12 File formats 29

12.1 Checkpoints 29
12.2 Tokenizers 29
12.3 Converting a HuggingFace model 29

13 Use-case cookbook 30

13.1 I want to see it work in five minutes 30
13.2 I want to run a real pretrained model 30
13.3 I want to teach a model my own task 30
13.4 I want a smaller, faster model to deploy 30
13.5 I want to train faster 30
13.6 I want the model to improve itself 31
13.7 I want to embed it in my application 31

14 Troubleshooting 32

14.1 Build problems 32
14.2 Runtime problems 32
14.3 GPU problems 32
14.4 Training problems 33
14.5 When something is genuinely wrong 33

15 Appendix 34

15.1 Environment variables 34
15.2 Exit codes 34
15.3 Quick command index 34
15.4 Where to read next 34
15.5 Design principles worth keeping 35

1 What this engine is

A GPT-2 / LLaMA-family transformer implemented from scratch in portable C11. There is no BLAS, no cuDNN, no CUDA toolkit, no Python at runtime, and no third-party header anywhere in the source tree. Every layer, the tokenizer, the optimizer, the checkpoint format, the agent loop and the GPU kernels are written here.

Two model families live side by side. The GPT-2 style path (model.c) uses LayerNorm, learned positional embeddings and GELU. The modern LLaMA-family path (llama.c) uses RMSNorm, rotary position embeddings, grouped-query attention and SwiGLU, with no biases on the output projections - that is the one that loads real models such as SmolLM2-135M and Qwen2.5.

1.1 The layout rule

Every C source file lives flat in the project root. There are no subdirectories for code and no other languages in the root. Helper scripts in other languages live in tools/ - that is the only directory allowed to hold them. Indentation is two spaces and every build must be -Wall clean under -std=c11.

1.2 Module map

File	Responsibility
`tensor.c`	Tensor struct, aligned allocation, gradient arrays
`matrix.c`	matmul and its backwards, plus a persistent thread pool; the single dispatch point for GPU offload
`activation.c`	softmax_rowwise, GELU
`layernorm.c / rmsnorm.c`	LayerNorm (GPT-2) and RMSNorm (LLaMA family), forward and backward
`rope.c`	Rotary position embeddings, half-split convention matching HuggingFace
`swiglu.c`	SwiGLU gated feed-forward network
`gqa.c`	Grouped-query causal attention
`attention.c`	GPT-2 multi-head attention and the classic transformer block
`llama.c`	LlamaBlock, LlamaModel, the LLMA/LLMQ checkpoints, and KV-cache decode
`llamatrain.c`	Cached-activation forward and full backward for the LLaMA path
`lora.c`	Low-rank adapters for fine-tuning a frozen base
`model.c`	The GPT model: embeddings, stacked blocks, output head, KV cache
`loss.c`	Softmax cross-entropy, plus pg_backward for policy gradients
`optim.c`	Adam / AdamW, gradient clipping, cosine schedule with warmup
`tokenizer.c`	Byte-level BPE: train, encode, decode, save, load
`hftok.c`	From-scratch byte-level BPE that reads a real HuggingFace vocabulary
`quant.c`	int8 per-channel weight quantization
`train.c`	The training driver: corpora, SFT masking, LoRA, checkpointing
`reward.c`	Verifiable reward for a sampled agent response
`grpo.c`	GRPO reinforcement learning from verifiable rewards
`gpu.c`	Optional OpenCL matmul offload, driver loaded at runtime
`agent.c / toolcall.c / toolexec.c / approval.c`	The tool-using agent: parse, approve, execute, observe
`selftest.c`	The 30-part smoke test that gates every change
`main.c`	CLI dispatch only

1.3 What it can and cannot do

It can: pre-train a small model from scratch, load and run real 135M-500M parameter checkpoints, fine-tune them with full backprop or LoRA, reinforce them with GRPO, quantize to int8, drive a multi-step tool-using agent, and generate text, images and audio.

> WARNING: It is not a competitor to llama.cpp or PyTorch on speed. The value here is that every line is readable and the whole pipeline is one static binary with no dependencies. Expect a few hundred GFLOP/s on a modern desktop CPU, and hours rather than minutes for a fine-tuning run.

2 Prerequisites and toolchain

The engine itself needs only a C11 compiler and libm. Everything else on this list is optional and only needed for a specific task.

Tool	Needed for	Notes
`C11 compiler`	everything	gcc, clang or zig cc. On Windows without gcc/clang, zig cc is the supported path.
`zig`	cross-compiling	Provides all target libcs; no per-OS toolchain needed. This is how the Colab binary is produced.
`make`	convenience	Optional. Every build is a single compiler invocation you can run by hand.
`python + numpy`	weight conversion, data prep	Only for tools/: converting HuggingFace checkpoints, building SFT corpora, evaluation.
`go`	webchat UI, this guide	tools/webchat (browser test UI) and tools/guidepdf (this document).
`An OpenCL driver`	GPU builds	Runtime only. Never needed to compile, not even to cross-compile a GPU binary.

2.1 Verifying the toolchain

```
zig version           # any recent version
gcc --version         # optional alternative
python --version      # 3.8+ for tools/
go version            # 1.21+ for tools/webchat and tools/guidepdf
```

2.2 Getting the sources

```
git clone <your-remote> llm-project
cd llm-project

# On a fresh machine, orient yourself first:
#   memory.md  - portable state snapshot, read this first
#   notes.md   - deep, dated engineering log
#   README.md  - reference manual
```

3 Building

There are three flavours: the default CPU build, an optional GPU build, and cross-compiled binaries for other platforms. The CPU build is the default and is what every other section of this guide assumes unless it says otherwise.

3.1 The CPU build (start here)

```
make                # -> ./transformer
./transformer       # runs the built-in self-test
make clean          # removes objects and the binary
```

Without make, a single command builds the whole engine. All three of these are equivalent:

```
gcc  -std=c11 -O3 -march=native -Wall -o transformer *.c -lm
clang -std=c11 -O3 -march=native -Wall -o transformer *.c -lm
zig cc -std=c11 -O2 -Wall -o transformer *.c -lm
```

On Windows PowerShell the glob needs expanding by hand:

```
$env:Path = "D:\env\zig;$env:Path"
$srcs = (Get-ChildItem *.c | ForEach-Object { $_.Name })
zig cc -std=c11 -O2 -Wall -o transformer.exe $srcs -lm
```

3.1.1 Compiler flags that matter

Flag	Effect
`-O3 / -O2`	Optimization. The matmul inner loops rely on the compiler auto-vectorizing; an -O0 build is roughly ten times slower.
`-march=native`	Lets the compiler emit AVX2/FMA for this exact CPU. Big win, but the binary may then crash on an older machine. Omit it for anything you distribute.
`-Wall`	Required. The tree is warning-clean and must stay that way.
`-std=c11`	Required. The code uses C11 features such as timespec_get.
`-DNDEBUG`	Compiles out check.h's shape and bounds assertions. They are ON by default because they abort at the fault site rather than corrupting memory quietly. Only define this once a workload is known-good and you want the last few percent.
`-pthread`	Needed on POSIX for the matmul thread pool. zig cc and Windows do not need it.

3.2 The GPU build

The GPU backend is optional and off by default. Turning it on adds gpu.c's OpenCL path; leaving it off compiles gpu.c to stubs, so the CPU build runs byte-for-byte the same code it always did and there is no conditional compilation anywhere outside gpu.c.

```
make GPU=1                              # native GPU build
zig cc -std=c11 -O2 -Wall -DTF_GPU -o transformer *.c -lm    # by hand

./transformer gpu-info                  # what did the backend find?
./transformer gpu-bench                 # is it actually faster than the CPU?
```

The GPU build requires no OpenCL SDK, no header, no import library and no CUDA toolkit.

The driver is opened with dlopen (POSIX) or LoadLibrary (Windows) at runtime and its twenty or so entry points are resolved by name. The handful of OpenCL types are declared inside gpu.c rather than taken from

<CL/cl.h>. That is what makes it possible to cross-compile a working GPU binary from a machine that has no GPU at all.

3.3 Cross-compiling

zig cc carries the libc for every target, so one machine builds them all.

```
make cross        # eight CPU targets into build/
make cross-gpu    # the Linux x86_64 GPU binary, for Colab
```

The eight CPU targets are x86_64-linux-gnu, aarch64-linux-gnu, x86_64-linux-musl, x86_64-windows-gnu, aarch64-windows-gnu, x86_64-macos, aarch64-macos and wasm32-wasi.

The GPU cross-build is a single command, and it is the artifact you upload to Colab:

```
zig cc -target x86_64-linux-gnu.2.31 -std=c11 -O2 -Wall -DTF_GPU \
  -o build/transformer-linux-x86_64-gpu *.c -lm
```

Piece	Why
`-target x86_64-linux-gnu.2.31`	Colab runs Ubuntu 22.04 (glibc 2.35). Pinning to 2.31 means the binary also runs on older distributions; pinning high would break on older ones.
`-DTF_GPU`	Compiles in the OpenCL backend. Safe even though the build machine has no GPU: the driver is only opened at runtime.
`wasm32-wasi excluded`	WebAssembly has no dlopen, so GPU builds are not produced for it.

From Windows, where there is no make, a script does all three:

```
powershell -File tools\build_gpu.ps1              # CPU + GPU + Colab binary
powershell -File tools\build_gpu.ps1 -Only colab  # just the Colab binary
```

3.4 Verifying a build

Running the binary with no arguments runs the self-test. It must print "Output validation: PASSED" and exit 0.

This is the gate every change in this project passes before it is committed. It covers thirty parts: tensor and matrix mathematics, every layer forward and backward with numerical gradient checks, the tokenizer round-trip, checkpoint save and load, sampling, KV-cache equivalence, int8 quantization, LoRA, gradient accumulation, the GRPO objective, the verifiable reward, and the GPU backend.

```
./transformer
...
[29] Policy gradient: gradient check + a closed GRPO loop
[30] GPU backend: results must match the CPU
Output validation: PASSED
```

> NOTE: Parts self-skip when their dependency is absent. On a CPU-only build part [30] prints "skipped (CPU-only build)" and does not affect the result, so the same test binary is valid everywhere.

4 Command reference

One binary, many subcommands. Running it with no arguments runs the self-test; an unrecognized subcommand prints usage and exits 1 rather than silently falling through.

Command	Purpose
`(no args)`	Run the built-in 30-part self-test
`train`	Train a GPT-2 style model on text
`llama-train`	Train the modern LLaMA-family architecture; also does SFT and LoRA
`dpo`	Preference learning: shown two answers and told which is better
`grpo`	Reinforcement learning from verifiable rewards
`generate`	Text generation from a GPTC checkpoint
`llama`	Generation from an LLMA or LLMQ checkpoint
`hfgen / hfchat`	Generation and chat using a real HuggingFace vocabulary, pure C
`agent`	The tool-using agent loop
`gpu-info / gpu-bench`	GPU backend status; CPU-versus-GPU timing
`imggen / audgen`	Autoregressive image and audio generation
`caption / text2media / t2i / vqvae`	Multimodal paths
`hftok`	Tokenizer inspection: encode and decode by hand

4.1 Global flags

These are recognized before the subcommand parses its own arguments and are stripped from the argument list, so they work with every command.

Flag	Effect
`--gpu`	Force the GPU path on (default when a GPU build finds a device).
`--no-gpu`	Force the CPU path even on a GPU build. This is how you A/B the same binary against itself honestly, rather than comparing two different builds.

4.2 generate, llama, hfgen, hfchat

```
transformer generate [model.bin] [tok.bin]
transformer generate --name mymodel
transformer llama <checkpoint.bin> [n_new=40] [tok.bin]
transformer hfgen <model.bin> <tok.hf> [n_new] [temp] [seed]
transformer hfchat <model.bin> <tok.hf> [n_new]
```

Command	Reads	Use it when
`generate`	GPTC checkpoint	You trained with `train`. One prompt per stdin line. Without a tokenizer it works in raw token ids.
`llama`	LLMA or LLMQ	You trained with `llama-train`. Auto-detects int8 by reading the magic. Without a tokenizer it reads space-separated ids and prints 'gen: <ids>'.
`hfgen`	LLMA/LLMQ + .hf vocab	You converted a real model. Text in, text out, entirely in C. Streams one JSON delta per token, which is what the webchat UI consumes.
`hfchat`	LLMA/LLMQ + .hf vocab	Same, but applies the model's own chat template (hftok_chat_render) so instruction-tuned models behave.

Sampling for `llama` is fixed at temperature 0.8, top-k 40, top-p 0.95, seed 1234. `hfgen` lets you pass temperature and seed as positional arguments.

4.3 agent

```
transformer agent [--goal "..."] [--cmd "<command>"] [--ask]

# drive it with a local model:
transformer agent --goal "create notes.txt containing hello and read it back" \
  --cmd "python tools/agent_local.py model/smollm/reason_e2.bin model/smollm/tok.hf"
```

Flag	Default	Meaning
`--goal`	a demo goal	The task given to the model.
`--cmd`	built-in script	A pluggable backend: the engine writes the transcript to this command's stdin and reads its next action from stdout. Without it a scripted demo runs, because the toy GPT cannot emit tool calls.
`--ask`	off (auto)	Prompt for approval before each tool call. run_command NEVER auto-approves regardless of this flag.

> WARNING: The tool executor is sandboxed to the current working directory: absolute paths, '..', drive letters and UNC paths are all rejected. Run the agent from a scratch directory anyway - it writes real files and runs real commands.

4.4 gpu-info and gpu-bench

```
transformer gpu-info        # is the backend compiled in? did it find a device?
transformer gpu-bench [reps]  # CPU vs GPU on this engine's real GEMM shapes
```

gpu-info distinguishes three states that are easy to confuse: not compiled in (a CPU-only build), compiled in but no device found (which is a correct, supported state - everything falls back to the CPU), and ready. gpu-bench reports GFLOP/s for both paths per shape, the numerical agreement between them, and a verdict telling you whether to use the GPU at all.

5 Preparing data

Two file formats cover everything. Which one you use decides whether the model learns to continue text or to follow instructions.

5.1 Plain text (pre-training)

A .txt file, or a directory that is walked recursively for .txt files. The whole corpus is tokenized and chopped into windows of --ctx tokens; the model learns to predict the next token at every position.

```
transformer train data/corpus.txt --steps 2000
transformer train data/mydocs/       # a whole directory tree
```

5.2 prompt TAB response (instruction tuning)

One example per line: the prompt, a single tab character, then the response. Passing --sft switches the trainer into masked mode, where loss is computed ONLY on the response tokens. The prompt is context the model conditions on, not something it is graded for reproducing.

```
What is the capital of France?\tParis.
create notes.txt containing hello\t<tool name="write_file" path="notes.txt">hello</tool>
```

> WARNING: The separator must be a real tab. A line without one is skipped silently, so a corpus saved by an editor that expands tabs into spaces trains on nothing at all.

5.3 Tools for building corpora

Script	What it does
`tools/make_sft.py`	Generate instruction/response pairs
`tools/make_qa.py`	Question-answer pairs
`tools/make_agent_traj.py`	Multi-step agent trajectories; --reason adds a <think> rationale before each action
`tools/crawl_wikipedia.py`	Grounded question-answer data from real articles
`tools/crawl_stackoverflow.py`	Real shell commands mined from answers
`tools/mix_sft.py`	Blend several corpora, so instruction tuning does not narrow the model catastrophically
`tools/clean_sft.py`	The quality gate: verify, deduplicate, balance
`tools/self_train.py`	STaR: harvest the agent's own verified-successful runs as new training rows
`tools/grpo_tasks.py`	Build a GRPO task file whose reference actions are all grounded

5.3.1 The quality gate

clean_sft.py sits between the generators and training. It rejects any pair whose tool arguments are not grounded in its own prompt (a path, a write body or a filename inside a command that the model would have to hallucinate), drops duplicates, and caps over-represented action types with full logging rather than silent truncation.

```
python tools/clean_sft.py --out data/clean.txt --report data/clean_report.txt \
    data/traj_reason.txt data/qa.txt data/toolcalls.txt
```

> TIP: Run this before every training run. Ungrounded pairs teach the model to invent filenames, and that failure mode is very hard to diagnose from a loss curve - the loss looks fine.

6 Training, step by step

6.1 Which trainer to use

Command	Architecture	Use when
`train`	GPT-2 style: LayerNorm, learned positions, GELU	Learning the codebase, or reproducing GPT-2 behaviour.
`llama-train`	RMSNorm, RoPE, GQA, SwiGLU, no biases	Anything real. This is the path that loads SmolLM2 and Qwen, and the only one with LoRA.

Both take the same flags. llama-train adds five of its own.

6.2 Walkthrough: train a model from scratch

1. Put a corpus at data/corpus.txt. A few hundred kilobytes of text is enough to see the loss fall; a few megabytes to get something coherent.
2. Start small and short, purely to check the plumbing. Sixty seconds of compute tells you whether the data loads and the loss moves.

```
transformer llama-train data/corpus.txt --steps 200 --d-model 128 --layers 4 \
  --heads 4 --ctx 128 --out model/test.bin --tok model/test.tok
```

3. Read the loss curve. It should fall steadily. If it is flat, the learning rate is too low or the data is not loading. If it goes to NaN or explodes, the learning rate is too high - and check that --clip is not 0.
4. Scale up. Increase --d-model, --layers and --steps together; a deeper model on too few steps underperforms a smaller one that finished.

```
transformer llama-train data/corpus.txt --steps 5000 --d-model 384 --layers 8 \
  --heads 6 --kv-heads 2 --ctx 512 --lr 3e-4 --warmup 100 --min-lr 3e-5 \
  --batch 8 --name mymodel
```

5. Generate from it and judge the output yourself.

```
transformer llama model/mymodel/model.bin 60 model/mymodel/tok.bin
```

6.3 Every training parameter

6.3.1 Output and data

Flag	Default	Meaning and when to change it
`--out`	model.bin	Where the weights are written.
`--tok`	tok.bin	Where the tokenizer is written (and, when resuming, read from).
`--name X`	-	Shorthand: writes model/X/model.bin and model/X/tok.bin. Overrides --out and --tok. Use it to keep runs tidy.
`--resume F`	off	Continue from an existing checkpoint instead of random init. Required for fine-tuning. The tokenizer must be the base model's - a fresh one assigns different ids to the same text, which makes the pretrained embeddings meaningless.
`--val-frac`	0.1	Fraction of tokens held out to report validation loss. Set 0 to train on everything; watch for the gap between train and validation widening, which is memorization.
`--checkpoint-every`	0	Save every N steps (0 = only at the end). Insurance on long runs. Ignored under LoRA, because merging mid-run would zero B and stall training.

6.3.2 Model geometry (fresh runs only; ignored when --resume is given)

Flag	Default	Meaning and when to change it
`--d-model`	128	Embedding width. The main capacity dial. Must be divisible by --heads.
`--layers`	3	Number of transformer blocks. Depth buys reasoning; cost is linear.
`--heads`	4	Attention heads. head_dim = d_model / heads, so keep head_dim at 32-128.
`--d-ff`	4 x d_model	Feed-forward width. Real LLaMA models use roughly 8/3 x d_model rather than 4x.
`--ctx`	64	Context window in tokens. Attention is O(ctx^2) in time and memory - this is the most expensive dial in the list.
`--merges`	300	BPE merges, so vocabulary = 256 + merges. Ignored when --hf-tok is used.
`--kv-heads`	= heads	LLaMA only. Grouped-query attention: fewer KV heads shrink the KV cache and speed decode. Must divide --heads evenly. 4:1 is typical.
`--rope-base`	10000	LLaMA only. RoPE frequency base. Raise it for longer contexts; must match the base model when fine-tuning.

6.3.3 The optimization recipe

Flag	Default	Meaning and when to change it
`--steps`	1000	Optimizer steps. With --batch B the model sees steps x B sequences.
`--lr`	3e-4	Peak learning rate. 3e-4 from scratch; 1e-5 to 5e-5 for full fine-tuning; 1e-3 to 5e-3 for LoRA, whose adapters start at zero and need a bigger push.
`--batch`	1	Gradient-accumulation micro-batches per step. Higher gives a far less noisy gradient. This is usually the best first change to a run that trains unstably.
`--clip`	1.0	Global gradient-norm clipping. ON by default and you should leave it on: an unclipped LoRA-SFT run in this project diverged at around step 210 and produced garbage. Pass 0 only to reproduce old unclipped measurements.
`--weight-decay`	0	AdamW decoupled decay, applied only to 2-D weight matrices, never to biases or norm gains. 0.01-0.1 helps a long run generalize.
`--warmup`	0	Linear warmup steps. Stabilizes the first steps while Adam's second moment is still cold. 100 or so is cheap insurance on any real run.
`--min-lr`	off	Cosine-decay floor. Setting it enables cosine decay from --lr down to this value. Without it the learning rate is constant.
`--seed`	1234	RNG seed. Fix it to make a run reproducible; vary it to check a result was not luck.

6.4 Supervised fine-tuning (SFT)

Add --sft and give the trainer a prompt-TAB-response file. Loss is computed only on response tokens; an EOS token is appended so the model learns where to stop.

```
transformer llama-train data/clean.txt --sft --resume model/base.bin \
  --tok model/base.tok --steps 900 --lr 2e-5 --batch 4 --clip 1.0 \
  --out model/sft.bin
```

> NOTE: Sequences are run at their true length rather than padded to the corpus maximum. This is bit-exact (causal attention never reads padding, and padded targets are -1, contributing no loss or gradient) but skips the wasted work - worth roughly 4-5x on a corpus of mostly short examples.

6.5 LoRA fine-tuning

LoRA freezes the base weights and learns a low-rank correction beside them: y = x.W + scale.(x.A).B. Only A and B are trained, so the optimizer state shrinks by orders of magnitude - which is what makes fine-tuning a real model feasible on a CPU. B starts at exactly zero, so an adapted model is bit-identical to its base before the first step.

```
transformer llama-train data/clean.txt --sft --resume model/base.bin \
  --hf-tok model/base/tok.hf --lora-r 8 --lora-alpha 16 \
  --steps 900 --lr 2e-3 --batch 4 --clip 1.0 --out model/lora.bin
```

Flag	Default	Meaning
`--lora-r`	0 (off)	Adapter rank. Above 0 the base freezes and only adapters train. 4-8 for style and format, 16-32 to teach genuinely new behaviour.
`--lora-alpha`	= r	Scaling: the correction is multiplied by alpha/r. Setting alpha = 2r is a common, well-behaved default.
`--hf-tok F`	off	Fine-tune using a real model's own vocabulary (a .hf file from tools/export_tokenizer.py) instead of training a byte-level BPE. Its real EOS is used, so no vocabulary slot is added.

Adapters are folded into the weights before saving, so the result is an ordinary LLMA checkpoint that needs no adapter support to run.

> WARNING: LoRA only pays off on a genuinely pretrained base. Adapting a toy model trained on a few hundred bytes reaches a much worse loss than a full fine-tune of the same data: frozen embeddings and a frozen output head simply cannot represent new tokens, and attention adapters alone cannot fix that.

6.6 Reading the output

```
LLaMA SFT (fine-tune): 1798 examples | vocab 49152 | seq len 332 | d_model=576 ...
  LoRA: rank 8 alpha 16 on Wq/Wk/Wv/Wo, base FROZEN (240 adapter tensors)
  variable-length batching: mean seq 96.4 / max 332 -> ~3.4x less padded work
  step     0/900  sft-loss 2.9184
  step    90/900  sft-loss 1.1043
```

Symptom	Likely cause and fix
`loss flat`	Learning rate too low, or the data did not parse. Check the example count in the header line is what you expect.
`loss -> NaN`	Learning rate too high, or --clip 0. Restore --clip 1.0 first, then halve the learning rate.
`loss falls then explodes`	A gradient spike. Add --warmup 100 and raise --batch.
`train falls, validation rises`	Memorization. Fewer steps, more data, or add --weight-decay.
`0 examples`	The prompt/response separator is not a real tab.

7 DPO: learning from preferences

Direct Preference Optimization (Rafailov et al.) is the third way this engine learns. Supervised fine-tuning is shown the right answer. GRPO acts and scores what it did. DPO is shown two answers and told which one is better.

7.1 Why it exists alongside GRPO

GRPO must SAMPLE to learn. DPO does not.

A GRPO step samples G rollouts per task, each a full KV-cache decode - 12 to 40 seconds on this CPU. Worse, most of those steps are then discarded: when every rollout in a group scores the same there is no advantage and no gradient, and on a model that already scores ~0.94 that is most groups.

DPO collects its preferences offline, once. Every training step is then a handful of forward passes over text that already exists, and no step is ever wasted.

7.2 The objective

```
s_w = log pi(y_w|x) - log pi_ref(y_w|x)     implicit reward, chosen
s_l = log pi(y_l|x) - log pi_ref(y_l|x)     implicit reward, rejected
h   = beta * (s_w - s_l)                    the preference margin
L   = -log sigmoid(h)
```

The paper's insight is that this IS the RLHF objective with the reward model solved for in closed form. That is why there is no reward model and no rollouts here - only a classification loss over pairs.

7.2.1 How it reaches the weights

The derivative of log pi(y_t) with respect to the logits is (onehot - p) - the same score function GRPO uses - so the entire backward is pg_backward again, with one scalar shared by every token of the pair:

```
g = beta * sigmoid(-h)
coeff = +g   on every scored token of the CHOSEN completion   (push up)
coeff = -g   on every scored token of the REJECTED completion (push down)
```

Notice what sigmoid(-h) does: it is near beta while the model still prefers the wrong answer, and near zero once the preference has been learned. The update anneals itself away, pair by pair.

7.2.2 The reference policy is free

With LoRA the base weights are frozen, so pi_ref is simply the same model with the adapters switched off - no second copy of a 621 MB checkpoint. That is why --lora-r must be greater than zero; without adapters there is no reference to be had without doubling memory, and the trainer refuses to start rather than silently doing something else.

> TIP: A correct run starts at loss 0.6931 (= ln 2) and margin 0.0000 exactly, because LoRA's B begins at zero, so at step 0 the policy IS the reference. If your run does not start there, the reference wiring is wrong before any training has happened.

7.3 Walkthrough

1. Build preference pairs from a corpus you have already verified.

```
python tools/dpo_pairs.py --out data/dpo_pairs.txt --max-chars 900 data/clean_reason.txt
```

2. Train. Watch the MARGIN - it starts near zero and should climb.

```
transformer dpo data/dpo_pairs.txt \
  --model model/smollm/reason_e2.bin --hf-tok model/smollm/tok.hf \
  --out model/smollm/dpo.bin --beta 0.1 --steps 400 --batch 4
```

3. Gate it. Re-measure with tools/remeasure.py and tools/probe3.py, and promote only if non-regressive.

7.4 Every DPO parameter

Flag	Default	Meaning
`--model / --hf-tok`	-	Required: the starting checkpoint and its own vocabulary.
`--beta`	0.1	KL strength - the one knob unique to DPO. It sets how far the policy may move from the reference. Lower drifts further and can degrade; higher stays cautious and learns less.
`--len-norm`	off	Divide each completion's log-prob by its token count. Standard DPO sums, which carries a known LENGTH BIAS: summing more negative terms makes long completions look worse regardless of quality. Turn on when chosen and rejected differ a lot in length.
`--steps / --batch`	200 / 4	Optimizer steps, and preference pairs accumulated per step.
`--lr`	5e-6	DPO is a sharp objective; SFT learning rates diverge here.
`--clip`	1.0	Gradient-norm clip. Leave it on.
`--lora-r / --lora-alpha`	8 / 16	Required (rank > 0). See the free-reference note above.
`--beta, --warmup, --min-lr, --weight-decay, --seed`	-	As in the other trainers.

7.5 Where the rejected side comes from

This is the whole game. The chosen side is easy - it is a response you have already verified. tools/dpo_pairs.py offers two sources for the rejected side:

Source	How	Trade-off
`Synthetic (default)`	Corrupt a verified response in the ways this model is known to fail: hallucinated filename, the measured ~5% character slip, empty write body, premature <done>, rambling past </tool>, wrong tool.	Cheap, deterministic, needs no sampling. But it can only teach the model to avoid defects somebody thought to encode.
`Real (--traces)`	Mine an AGENT_TRACE JSONL for responses that actually FAIL clean_sft.verify(), and pair each with the correct response for the same prompt.	Strictly better data - on-policy mistakes rather than imagined ones. But you only get as many as the model has actually made.

> WARNING: DPO learns WHATEVER separates the two sides. An early version of the generator corrupted a reasoning-stripped copy while writing the untouched response as the chosen side, so every pair differed in two ways - the defect, and the presence of a <think> span. That corpus would have quietly taught the model to prefer responses containing <think> rather than to stop rambling. Corruptions are now applied to the FULL chosen response. Read your generated pairs; no test catches this.

7.6 Measured cost, and the honest comparison

Phase	Cost on SmolLM2-135M
`Reference pass`	~1.2 s per pair, ONE time (~29 min for 1,420 pairs), then cached - pi_ref never changes.
`Training`	~18 s per step at --batch 4.

Per step that is comparable to GRPO, not dramatically faster. The win is that no step is discarded - in testing GRPO skipped two or three of every four groups - plus determinism and no decode loop to wait on.

7.7 Honest expectations

A falling loss and a rising margin mean the PREFERENCE was learned. They do not mean the model got better at the task.

Only tools/remeasure.py can tell you that, which is why promotion is gated on it. If the margin rises but re-measurement does not improve, the most likely cause is the synthetic negatives: the model has learned to avoid corruptions it was never going to make. Collect a real trace and re-run tools/dpo_pairs.py with --traces.

8 GRPO: reinforcement learning from verifiable rewards

Every other trainer here is supervised: it is shown the right answer and pushed toward it. GRPO instead lets the model ACT, scores what it did against a reward that can be checked mechanically, and pushes it toward its own better-than-average attempts and away from its worse-than-average ones.

8.1 How it works

1. For one task, sample G rollouts from the current policy, using KV-cache decode with the LoRA adapters applied - so the samples really do come from the policy being updated.
2. Score each rollout with the verifiable reward: is it exactly one well-formed action, are its arguments grounded in the prompt, does it match the reference action.
3. Normalize the rewards WITHIN the group: A_i = (r_i - mean) / (std + 1e-4). This is the 'group relative' part, and it is what replaces a value network - the group's own mean is the baseline.
4. Take one clipped policy-gradient step, optionally pulled toward a reference policy by a KL penalty.

GRPO learns from variance. If every rollout in a group scores the same, every advantage is zero and the group teaches nothing - it is skipped before any forward pass.

That is the honest limit of the method. Where the model is uniformly right or uniformly wrong there is no signal at all. The realistic win is sharpening the cases it gets right only sometimes, and actively suppressing confidently-wrong ones - which rejection sampling (the STaR loop in tools/self_train.py) cannot do, because it throws failures away.

8.2 Walkthrough

1. Build a task file. It is the same prompt-TAB-response format the SFT trainer reads; the response column becomes the reference action the reward scores against.

```
python tools/grpo_tasks.py --out data/grpo_tasks.txt --actions-only \
    --max-chars 900 data/clean_reason.txt
```

> WARNING: A bad row in SFT data teaches the model one wrong answer. A bad row in an RL task file teaches the REWARD to lie - the model then earns credit for reproducing a hallucination. grpo_tasks.py puts every reference through clean_sft.verify() and drops what it rejects.

2. Run a short job first and time it. Rollout cost is G x max-new decode steps per task, so measure before committing to hours.

```
transformer grpo data/grpo_tasks.txt \
  --model model/smollm/reason_e2.bin --hf-tok model/smollm/tok.hf \
  --out model/smollm/grpo.bin --group 8 --steps 4 --verbose
```

3. Read the diagnostics: how many groups were flat (no spread, no signal), and whether rollouts are hitting the --max-new cap instead of stopping themselves.
4. Run for real, then gate the result. Re-measure with tools/remeasure.py and tools/probe3.py and promote the new checkpoint ONLY if it is non-regressive.

8.3 Every GRPO parameter

8.3.1 Required

Flag	Meaning
`--model F`	The LLMA checkpoint to start from - the policy being improved.
`--hf-tok F`	That model's own .hf vocabulary.

8.3.2 Rollouts

Flag	Default	Meaning
`--group`	8	G, the rollouts per task. Must be at least 2 - a group of one has no mean to be relative to. Larger groups give a better baseline and more chance of spread, at linear cost.
`--max-new`	96	Cap on sampled tokens per rollout. Too small truncates the action mid-emission, every rollout then scores identically, and the gradient vanishes. The run warns when more than a quarter of rollouts hit the cap.
`--temp`	1.0	Sampling temperature - the exploration dial. Too low and every rollout in a group is identical, so nothing is learned. Raise it when the run reports many flat groups.
`--top-k / --top-p`	0 / 0.95	Nucleus sampling bounds applied while exploring.

8.3.3 Optimization

Flag	Default	Meaning
`--steps`	100	Optimizer steps.
`--groups`	1	Task groups accumulated into one optimizer step, like --batch for SFT.
`--inner-epochs`	1	How many updates to take per set of rollouts. Above 1 reuses the expensive rollouts; the ratio clip is what keeps that honest.
`--lr`	1e-5	Learning rate. RL wants far less than SFT - the gradient is much noisier.
`--clip`	1.0	Gradient-norm clip. Leave it on; an RL gradient is far spikier than a supervised one.
`--lora-r / --lora-alpha`	8 / 16	LoRA rank and scaling. 0 means a full fine-tune, which is not recommended here.
`--weight-decay, --warmup, --min-lr`	0, 0, off	As in the supervised trainer.

8.3.4 The objective

Flag	Default	Meaning
`--clip-eps`	0.2	PPO-style ratio clip, 1 +/- eps. Removes gradient that would push a token further in a direction it has already moved too far. 0 disables clipping.
`--kl`	0 (off)	KL penalty pulling the policy toward a reference. The reference is the FROZEN BASE - the same weights with the adapters switched off - so it costs one extra forward per rollout and no extra memory. Raise it if the model drifts into degenerate text.
`--seed`	1	RNG seed.
`--verbose`	off	Print one scored sample rollout per group, with the reward broken down by term.

8.4 The reward

Three terms, each checkable against something outside the model. The model's own opinion of its work is never consulted.

Term	Weight	What it checks
`format`	0.20	Exactly one well-formed, properly closed <tool> or <done> block, with nothing trailing.
`grounded`	0.30	Every path, write body and filename inside a command occurs in the PROMPT as a whole token. This is what catches a hallucinated or character-slipped filename.
`match`	0.50	The parsed action equals the reference action. Partial credit for the right tool with the wrong argument.

> TIP: Watch the --verbose breakdown for reward hacking: a rising total while the match term stays flat means the model found a cheap way to satisfy format and grounding without actually doing the task.

8.5 Two behaviours worth knowing

Rollouts stop at a closing </tool> or </done>, not only at EOS. The deployed agent parses the first block and executes only that, so text after it is discarded at runtime - scoring or training on tokens the harness throws away would optimize something that does not exist in deployment. In practice this also cuts rollout length substantially, because a small model keeps talking long after it is done.

A group with no spread is skipped before any forward pass, and the run counts them. Many flat groups means either the task set is too easy for the model or the temperature is too low.

9 GPU acceleration

The GPU backend offloads matmul and its two backward forms - essentially all the arithmetic in both inference and training - through an OpenCL driver loaded at runtime. It is optional, off unless compiled with -DTF_GPU, and falls back to the CPU whenever it cannot help.

9.1 How it hooks in

matmul in matrix.c was already a small dispatcher and is the single chokepoint for all forty-nine call sites across the engine, so the hook is one line and no call site changed:

```
void matmul(const Tensor *A, const Tensor *B, Tensor *C) {
  tf_check_matmul(A, B, C);
  const int M = A->shape[0], K = A->shape[1], N = B->shape[1];
  if (gpu_matmul(A->data, B->data, C->data, M, K, N)) return;  /* GPU took it */
  ... the unchanged CPU path ...
}
```

gpu_matmul returns 0 - meaning 'not mine' - for a CPU-only build, when no device was found, when --no-gpu is in force, or when the problem is too small to be worth the dispatch.

9.2 When the GPU is used

Env var	Default	Meaning
`TF_GPU_MIN_M`	8	Minimum rows M. Single-token decode is M=1 and memory-bandwidth-bound: uploading a 113 MB output head to multiply one row by it is far slower than doing it on the CPU.
`TF_GPU_MIN_MACS`	4194304	Minimum M x N x K. Below this the dispatch and transfer cost dominate the arithmetic.
`TF_THREADS`	CPU count	Threads in the CPU matmul pool. Also useful for making CPU-versus-GPU comparisons fair.

Both GPU thresholds are read from the environment, so they can be retuned on a new device without rebuilding.

Text generation will not get faster. Decode is M=1 and stays on the CPU by design; training and prefill are what the GPU path is for.

9.3 Deciding whether to use it

A GPU being present does not mean it is worth using. Run the benchmark and read the verdict:

```
transformer gpu-info
transformer gpu-bench 5
```

Measured on the development machine - an Intel UHD 730 integrated GPU (24 execution units, sharing the CPU's memory bus) against a 10-core i5-14400:

Shape (M x K x N)	CPU	GPU	Ratio
`256 x 576 x 1536`	148 GFLOP/s	66 GFLOP/s	0.45x
`256 x 1536 x 576`	212 GFLOP/s	60 GFLOP/s	0.28x
`512 x 576 x 576`	269 GFLOP/s	68 GFLOP/s	0.25x
`256 x 576 x 49152`	41 GFLOP/s	53 GFLOP/s	1.30x

> WARNING: On that hardware the GPU is SLOWER: 0.46x overall. An integrated GPU losing to a good multi-core CPU is the normal result - it shares the same memory bus and has a fraction of the arithmetic units. Use --no-gpu there. The GPU build exists for DISCRETE GPUs.

The verdict is summarized as a geometric mean of the per-shape ratios rather than a ratio of total times. Summing absolute times lets one huge shape decide the answer: the output-head GEMM alone is about a hundred times the work of a layer projection, so an arithmetic summary reported a misleading speedup while the GPU was in fact losing on the three shapes a transformer layer actually runs - and a layer's projections execute once per layer per forward, against the output head's once.

9.4 Correctness

Self-test part [30] compares GPU against CPU on deliberately ragged shapes - none of them a multiple of the kernel's 16-wide tile, and one not a multiple of the forward kernel's 64-column block - because the boundary tile is where a tiled GEMM classically breaks and it hides completely on tidy dimensions. It also checks the backward kernels ACCUMULATE into pre-seeded gradients rather than overwriting them: a kernel that stores instead of accumulating passes a zero-initialized test and then silently discards every previously accumulated gradient in real training.

Quantity	Agreement with the CPU
`forward matmul`	bit-identical (0.0e+00)
`dB (weight gradient)`	bit-identical (0.0e+00)
`dA / dx (input gradient)`	2.0e-07 - the CPU reassociates that reduction to vectorize, as matrix.h documents, so the last bits legitimately differ

9.5 Known limitation

Every operand is uploaded on every call. Caching weight CONTENTS would be worth roughly twice the throughput, but it cannot be done safely without an explicit immutability hint from the model code: activations are rewritten between matmuls by RMSNorm, RoPE and SwiGLU, and attention.c passes ACTIVATIONS as the B operand for Q times K-transpose, so 'B is a weight' is not a safe assumption. Guessing wrong would not crash - it would silently train on stale weights.

10 Running on Google Colab

Colab gives you a discrete NVIDIA GPU on Linux x86_64, which is what the GPU backend is actually for. The workflow is: cross-compile on your machine, upload one file, run.

10.1 Step 1 - build the binary

```
make cross-gpu                                    # Linux / macOS
powershell -File tools\build_gpu.ps1 -Only colab   # Windows

# -> build/transformer-linux-x86_64-gpu  (about 1.1 MB)
```

This binary is also committed to the repository, so you can download it directly rather than building it. It hard-links only libc, libdl, libm and libpthread; the three OpenCL library names it can use are embedded as strings for dlopen, not as link-time dependencies. That is why it runs whether or not an OpenCL driver is installed.

10.2 Step 2 - open the notebook and pick a GPU

1. Go to colab.research.google.com and choose File, then Upload notebook.
2. Upload tools/colab_gpu_test.ipynb.
3. Choose Runtime, then Change runtime type, then T4 GPU, then Save.

> WARNING: Change the runtime type BEFORE uploading any files. Switching runtimes restarts the virtual machine and deletes everything you have uploaded.

10.3 Step 3 - upload the binary

Run the notebook cells in order. The upload cell calls files.upload() and opens a Choose Files button; pick build/transformer-linux-x86_64-gpu.

The alternative, often quicker: click the folder icon in the left sidebar and drag the file into the file list. If you do that, replace the upload cell with:

```
BIN = './transformer-linux-x86_64-gpu'
import os; os.chmod(BIN, 0o755)
```

> NOTE: Uploaded files live on a temporary virtual machine and disappear when the session ends or is reset. You will re-upload each session.

10.4 Step 4 - install an OpenCL loader

Colab ships the NVIDIA driver, which provides OpenCL through libnvidia-opencl.so.1, but usually not the vendor-neutral loader or the file that points it at NVIDIA. Both are one line:

```
!apt-get -qq install -y ocl-icd-libopencl1 clinfo
!mkdir -p /etc/OpenCL/vendors && echo libnvidia-opencl.so.1 > /etc/OpenCL/vendors/nvidia.icd
!clinfo -l
```

Skipping this still works - the backend falls back to opening the NVIDIA library directly - but installing the loader is the supported path and lets clinfo corroborate what the engine reports.

10.5 Step 5 - verify, then measure

```
!chmod +x transformer-linux-x86_64-gpu
!./transformer-linux-x86_64-gpu gpu-info
!./transformer-linux-x86_64-gpu 2>&1 | tail -n 12     # the self-test
!./transformer-linux-x86_64-gpu gpu-bench 5
```

Cell	What you should see
`nvidia-smi`	Tesla T4 (or L4, A100)
`clinfo -l`	Platform #0: NVIDIA CUDA
`gpu-info`	status: READY, with an NVIDIA device name and its compute-unit count
`the self-test`	Output validation: PASSED, with part [30] green
`gpu-bench`	GFLOP/s per shape, agreement with the CPU, and a verdict line

Part [30] passing is the real deliverable. Read the timings only after it is green - a fast wrong answer is not a speedup.

10.6 Step 6 - a real workload

--no-gpu forces the CPU path in the same binary, which is what makes this an honest comparison rather than two different builds.

```
!./transformer-linux-x86_64-gpu llama-train data/bench.txt --steps 20 \
    --d-model 256 --layers 4 --heads 8 --ctx 128 --out /tmp/g.bin --tok /tmp/g.tok --gpu
!./transformer-linux-x86_64-gpu llama-train data/bench.txt --steps 20 \
    --d-model 256 --layers 4 --heads 8 --ctx 128 --out /tmp/c.bin --tok /tmp/c.tok --no-gpu
```

> NOTE: Colab's CPU is typically only two virtual cores, so the CPU column there is much weaker than a real desktop. The meaningful comparison for 'should I train on Colab' is Colab's GPU column against your own machine's CPU column.

11 Loading a model in your own program

The engine is a set of plain C modules with no global initialization, so linking it into your own program is a matter of including two headers and calling four functions.

11.1 The pieces you need

Header	Gives you
`llama.h`	LlamaModel, llama_model_load, LlamaKV, llama_decode_step, and the int8 QLlamaModel equivalents
`hftok.h`	HFTok, hftok_load, hftok_encode, hftok_decode, hftok_is_eog, hftok_chat_render
`sampler.h`	sample_token (temperature, top-k, top-p) and sample_argmax, with a seeded RNG
`tensor.h`	The Tensor struct that logits come back in

11.2 The four steps

1. Load the vocabulary with hftok_load, and the weights with llama_model_load. Both return 0 on success and validate their headers before allocating, so a corrupt or hostile file is rejected rather than acted on.
2. Create a KV cache with llama_kv_create. This is what makes generation O(n) per token instead of O(n^2): each new token computes only its own key and value and attends over the cache.
3. Prefill - feed every prompt token through llama_decode_step. After the last one, the logits describe the distribution over the NEXT token.
4. Sample a token, append it to the cache by decoding it, and repeat until hftok_is_eog says stop or you hit your own limit.

11.3 A complete, working example

This is a whole program. It loads a model, encodes a prompt, generates, and prints text.

```c
// demo.c - generate text with the engine, in about 50 lines.
#include <stdio.h>
#include <stdlib.h>
#include "llama.h"
#include "hftok.h"
#include "sampler.h"

int main(int argc, char **argv) {
  if (argc < 4) { fprintf(stderr, "usage: demo <model.bin> <tok.hf> <prompt>\n"); return 2; }

  /* 1. Load the vocabulary and the weights. */
  HFTok tok;
  if (hftok_load(&tok, argv[2]) != 0) { fprintf(stderr, "bad tokenizer\n"); return 1; }
  LlamaModel m;
  if (llama_model_load(argv[1], &m) != 0) { fprintf(stderr, "bad model\n"); return 1; }

  /* 2. Encode the prompt. */
  int ids[2048];
  int n = hftok_encode(&tok, argv[3], ids, 2048);

  /* 3. KV cache + a one-row logits tensor. */
  LlamaKV kv = llama_kv_create(&m);
  int shape[2] = { 1, m.vocab };
  Tensor logits = tensor_create_nograd(shape, 2);
  Rng rng = rng_create(1234);

  /* Prefill: after the last prompt token, logits predict the next one. */
  for (int i = 0; i < n; i++) llama_decode_step(&m, &kv, ids[i], &logits);

  /* 4. Sample, decode one piece at a time, feed it back. */
  char piece[256];
  for (int i = 0; i < 100 && kv.len < m.max_seq; i++) {
    int t = sample_token(&logits, 0, 0.8f, 40, 0.95f, &rng);
    if (hftok_is_eog(&tok, t)) break;
    hftok_decode(&tok, &t, 1, piece, sizeof(piece));
    fputs(piece, stdout);
    fflush(stdout);
    llama_decode_step(&m, &kv, t, &logits);
  }
  putchar('\n');

  tensor_free(&logits);
  llama_kv_free(&kv);
  llama_model_free(&m);
  hftok_free(&tok);
  return 0;
}
```

11.3.1 Building it

Compile your file together with the engine sources, excluding main.c so there is only one entry point:

```
# Linux / macOS
gcc -std=c11 -O3 -Wall -o demo demo.c \
    $(ls *.c | grep -v '^main\.c$') -lm -pthread

# Windows PowerShell, with zig
$srcs = (Get-ChildItem *.c | Where-Object { $_.Name -ne 'main.c' } | ForEach-Object { $_.Name
})
zig cc -std=c11 -O2 -Wall -o demo.exe demo.c $srcs -lm

# with the GPU backend
gcc -std=c11 -O3 -Wall -DTF_GPU -o demo demo.c \
    $(ls *.c | grep -v '^main\.c$') -lm -pthread -ldl
```

> TIP: You do not need every module. A generation-only program needs tensor, matrix, activation, rmsnorm, rope, swiglu, gqa, quant, llama, lora, sampler and hftok. Linking them all is simpler and costs about a megabyte.

11.4 Memory and performance notes

Concern	What to do
`Model memory`	An fp32 135M model is roughly 540 MB. llama_model_load uses tensor_create_nograd, so no gradient arrays are allocated - that alone halves what a training build would use.
`Halving it again`	Convert to int8 with tools/convert_llama.py --int8 and load with q_llama_model_load, then decode with q_llama_decode_step. About 4x less memory and less memory traffic.
`Threads`	The matmul pool starts lazily on first use and sizes itself from the CPU count. Override with the TF_THREADS environment variable.
`Context length`	kv.len must stay below m.max_seq. Check it in your loop, as the example does; overrunning is a bounds error, not a graceful truncation.
`Thread safety`	The thread pool and the GPU backend are process-global. Drive one model from one thread, or serialize access.

11.5 Chat models

An instruction-tuned model expects its own chat template. hftok_chat_render applies the one stored in the .hf file, including the model's default system prompt when it has one:

```
const char *roles[]    = { "system", "user" };
const char *contents[] = { "You are a helpful assistant.", "Why is the sky blue?" };
char rendered[8192];
hftok_chat_render(&tok, roles, contents, 2, rendered, sizeof(rendered));
int n = hftok_encode(&tok, rendered, ids, 2048);
/* ...then prefill and sample exactly as above... */
```

> WARNING: Skipping the chat template is the most common reason an instruction-tuned model produces rambling continuations instead of answers. The weights are fine; it simply never saw that prompt shape in training.

12 File formats

12.1 Checkpoints

Magic	Format	Written by / read by
`GPTC`	GPT-2 style, fp32	transformer train; read by generate and gpt_load
`LLMA`	LLaMA family, fp32	transformer llama-train and grpo; read by llama, hfgen and llama_model_load
`LLMQ`	LLaMA family, int8 per-channel	tools/convert_llama.py --int8; read by llama and q_llama_model_load. Loaded without dequantizing, so it stays about 4x smaller in memory.

All three begin with a 4-byte magic and a version, then a fixed configuration block, then every weight in a canonical order shared by the writer and the reader. Loaders validate the header before allocating anything: non-positive or absurd dimensions, a zero KV-head count, grouped-query heads that do not divide evenly, and weight element counts that would overflow are all rejected. Saves are atomic - written to a temporary file and renamed - so an interrupted save cannot destroy a good checkpoint.

> NOTE: Checkpoints are little-endian and not endianness-converted on load. They are portable across x86 and ARM, which are both little-endian, but not to a big-endian machine.

12.2 Tokenizers

File	What it is
`.tok` / `tok.bin`	A byte-level BPE trained by this engine on your corpus.
`.hf`	A real model's vocabulary, exported once by tools/export_tokenizer.py. At runtime the parsing is pure C - hftok.c implements byte-level BPE from scratch, including the GPT-2 pretokenization regular expression by hand.

The insight that makes one implementation serve both: HuggingFace BPE ids are in merge order, so the tiktoken minimum-combined-id merge rule works for both tiktoken and HuggingFace vocabularies. Exports have been verified exact against the reference implementations for GPT-2, SmolLM2 and Qwen2.5 on punctuation, code, numbers and contractions.

```
python tools/export_tokenizer.py gpt2 model/gpt2/tok.hf
python tools/export_tokenizer.py HuggingFaceTB/SmolLM2-135M model/smollm/tok.hf
```

12.3 Converting a HuggingFace model

```
python tools/convert_llama.py <hf-repo-or-path> model/name/model.bin
python tools/convert_llama.py <hf-repo-or-path> model/name/model.bin --int8
python tools/export_tokenizer.py <hf-repo> model/name/tok.hf

transformer hfchat model/name/model.bin model/name/tok.hf 100
```

> WARNING: Diffusion models (Stable Diffusion, FLUX) cannot be loaded. They are not autoregressive transformers. The text-to-image path here is the autoregressive DALL-E/Parti approach instead - see the t2i and vqvae subcommands.

13 Use-case cookbook

Recipes for the things people actually want to do, in the order most people want them.

13.1 I want to see it work in five minutes

```
make && ./transformer                      # self-test: Output validation: PASSED
transformer train data/sample.txt --steps 300 --name toy
echo 'hello' | transformer generate --name toy
```

13.2 I want to run a real pretrained model

```
pip install torch transformers        # once, for the converter only
python tools/convert_llama.py HuggingFaceTB/SmolLM2-135M-Instruct model/smol/model.bin
python tools/export_tokenizer.py HuggingFaceTB/SmolLM2-135M-Instruct model/smol/tok.hf
transformer hfchat model/smol/model.bin model/smol/tok.hf 120
```

> TIP: Convert the -Instruct variant, not the base one. The base model has no chat behaviour to draw on and will simply continue your prompt.

13.3 I want to teach a model my own task

1. Write examples as prompt-TAB-response, a few hundred at minimum, a few thousand for real generalization.
2. Run them through the quality gate, and read the report it prints.

```
python tools/clean_sft.py --out data/clean.txt --report data/report.txt data/mine.txt
```

3. LoRA fine-tune from a real base. Two epochs over the data usually beats one long one.

```
transformer llama-train data/clean.txt --sft --resume model/smol/model.bin \
  --hf-tok model/smol/tok.hf --lora-r 8 --lora-alpha 16 \
  --steps 900 --lr 2e-3 --batch 4 --clip 1.0 --out model/mine.bin
```

4. Measure before you trust it. Hold out examples the model has never seen and check it generalizes rather than memorizes.

13.4 I want a smaller, faster model to deploy

```
python tools/convert_llama.py model/mine.bin model/mine_int8.bin --int8
transformer llama model/mine_int8.bin 60 model/mine.tok
```

int8 cuts memory roughly fourfold and reduces memory traffic, which is what decode is actually limited by. Quality loss is small because quantization is per-channel.

13.5 I want to train faster

Lever	Effect
`--batch 8`	Much less gradient noise per step; often the single best change.
`TF_THREADS`	Confirm the thread pool is using every core.
`-march=native`	Lets the compiler use AVX2/FMA on this machine.
`--lora-r 8`	Trains far fewer parameters; the optimizer state shrinks by orders of magnitude.

Lever	Effect
`Shorter --ctx`	Attention is quadratic in context length. Halving it is close to a 4x saving.
`A discrete GPU`	Build with GPU=1 and benchmark. On an integrated GPU expect to lose; see section 8.

13.6 I want the model to improve itself

Two mechanisms, in increasing order of power.

Approach	How it works	Limitation
`STaR (tools/self_train.py)`	Trace real agent runs, keep the verified-successful ones, fold them back in as new supervised rows.	Discards failures, so a failed rollout teaches nothing.
`DPO (transformer dpo)`	Show it two answers and say which is better. No sampling at training time - the preferences are collected offline.	You must supply the pairs, and it learns whatever separates them - including differences you did not intend.
`GRPO (transformer grpo)`	Sample a group of rollouts, score them, and push toward the better-than-average while suppressing the worse.	Needs variance within a group; a uniformly right or wrong model gives no signal.

> WARNING: Never train on self-generated data alone - always mix in the original corpus, or the distribution narrows and the model degrades. And never promote a checkpoint without re-measuring against the one it replaces.

13.7 I want to embed it in my application

See section 10 for a complete working program. In short: include llama.h and hftok.h, call llama_model_load and hftok_load, create a KV cache, prefill, then sample in a loop. Compile your file alongside the engine sources with main.c excluded.

14 Troubleshooting

14.1 Build problems

Symptom	Cause and fix
`implicit declaration of function`	A missing #include. Every module declares its API in the matching .h.
`undefined reference to matmul`	You compiled one file instead of all of them. Pass every .c.
`undefined reference to main`	You included main.c alongside your own main. Exclude it.
`undefined reference to dlopen`	A GPU build on POSIX needs -ldl.
`illegal instruction at startup`	A -march=native binary running on an older CPU. Rebuild without it.
`warnings anywhere`	Treat them as errors; this tree is warning-clean by policy.

14.2 Runtime problems

Symptom	Cause and fix
`Error: cannot open '<file>'`	Wrong path, or a relative path from the wrong working directory.
`not a v1 GPTC checkpoint`	Format mismatch: `generate` reads GPTC, `llama` reads LLMA and LLMQ. Check which trainer produced it.
`model vocab < tokenizer vocab`	The checkpoint and the tokenizer are from different runs. Fine-tuning must reuse the base model's tokenizer.
`SFT file has no prompt/response lines`	The separator is not a real tab character.
`sequences need N positions but ctx is M`	Your examples are longer than the base model's context. Shorten them or raise --ctx on a fresh model.
`Generation is gibberish`	For an instruction-tuned model, apply the chat template (hftok_chat_render). Otherwise the model may simply be undertrained.
`Generation never stops`	No EOS in the vocabulary, or the model never learned to emit it. SFT appends EOS; pre-training does not.
`A run seems to hang`	Attention is O(seq^2) on the CPU. Media and long-context runs are slow, not stuck. Watch for progress lines - the trainer flushes stdout so a redirected log stays live.

14.3 GPU problems

Symptom	Cause and fix
`NOT COMPILED IN`	A CPU-only build. Rebuild with make GPU=1 or -DTF_GPU.
`no OpenCL library found`	No ICD loader installed. On Linux: apt-get install ocl-icd-libopencl1.
`no platform is registered`	The loader is there but no vendor driver. On Colab, write /etc/OpenCL/vendors/nvidia.icd as shown in section 9.
`platform present but no GPU device`	An OpenCL runtime exists but exposes only a CPU device. Install your GPU vendor's compute runtime.
`GPU slower than CPU`	Expected on an integrated GPU. Run gpu-bench, read the verdict, use --no-gpu.
`falling back to the CPU mid-run`	A device error. The backend disables itself rather than aborting, so a long run degrades instead of dying. The message names the operation that failed.

14.4 Training problems

See the table at the end of section 6 for loss-curve symptoms. The two that catch people most often: --clip 0 makes an otherwise-fine run diverge, and a corpus whose tabs were expanded into spaces silently trains on zero examples.

14.5 When something is genuinely wrong

1. Run the self-test. It exercises every layer with numerical gradient checks and will usually localize the fault to one part.
2. Rebuild without -DNDEBUG. The assertions in check.h abort at the fault site with the actual shapes, instead of letting a bad index corrupt memory quietly.
3. Reduce to the smallest case that still fails - a tiny model, two examples, three steps.
4. Check notes.md. It is a dated engineering log and this class of problem may already be recorded there with its fix.

15 Appendix

15.1 Environment variables

Variable	Default	Effect
`TF_THREADS`	CPU count	Threads in the matmul pool. Set 1 to force single-threaded, which makes profiling comparable.
`TF_GPU_MIN_M`	8	Minimum rows before the GPU takes a matmul.
`TF_GPU_MIN_MACS`	4194304	Minimum M x N x K before the GPU takes a matmul.
`AGENT_TRACE`	off	When set to a path, every agent step appends its exact prompt/response pair as JSONL - the input to the STaR loop.

15.2 Exit codes

Code	Meaning
`0`	Success. For the self-test, all parts passed.
`1`	A runtime error: a file could not be opened, a checkpoint was invalid, or the self-test failed.
`2`	Usage error: a missing argument or an unknown flag. Usage was printed.

15.3 Quick command index

```
# build
make                                  # CPU
make GPU=1                            # + OpenCL
make cross-gpu                        # Linux binary for Colab

# verify
./transformer                         # self-test; must print PASSED
./transformer gpu-info                # GPU backend status
./transformer gpu-bench 5             # CPU vs GPU, with a verdict

# train
transformer llama-train DATA --steps N --d-model D --layers L --name X
transformer llama-train DATA --sft --resume BASE --lora-r 8 --out OUT
transformer grpo TASKS --model M --hf-tok V --group 8 --steps N --out OUT

# run
transformer llama MODEL 60 TOK
transformer hfchat MODEL TOK 120
transformer agent --goal "..." --cmd "python tools/agent_local.py M T"
```

15.4 Where to read next

File	What it holds
`README.md`	The reference manual: every subcommand, the C API, the coding rules.
`memory.md`	A portable state snapshot, committed to the repository so a fresh machine can catch up quickly. Read this first on a new checkout.
`notes.md`	A dated engineering log: what was tried, what was measured, what failed and why.
`tools/README.md`	What each helper script does.
`selftest.c`	Thirty worked examples of the API, each with its correctness criterion spelled out.

15.5 Design principles worth keeping

- Every step ends green. The self-test must print PASSED before anything is committed.
- Correctness is proved numerically. New backward passes are gradient-checked against finite differences, judging well-conditioned entries, because float32 has a noise floor.
- Compare tensors as max-difference over max-value, not per-element relative error. A value near zero makes per-element relative error meaningless.
- Summarize ratios with a geometric mean. Summing absolute times lets one large case decide the verdict.
- Never promote a model without re-measuring it against the one it replaces.
- Prefer being honest about a limitation over hiding it. Every performance claim in this guide has a measurement behind it, including the ones that are unflattering.